

Front-End Performance

Devotopia — measurement, diagnosis and fixes

Playwright was added to the repository and used to measure the site before any code was changed. Three causes were found and fixed. Every number in this report comes from the production build, not the dev server.

1. Summary

The site was not slow because of the server. Time to first byte measured 3–9 ms across every route. The cost was JavaScript: pages shipped up to 316 kB before they could become interactive, and the heaviest of it was never needed to paint the screen.

Three changes removed that weight. The result is a 49% reduction on the heaviest route and a Largest Contentful Paint on the sign-up pages that dropped from 1.54 seconds to under 0.3.

Headline results

Measure	Before	After	Change
Dashboard first-load JS	316 kB	161 kB	– 49%
Quiz first-load JS	243 kB	131 kB	– 46%
Auth pages first-load JS	154 kB	130 kB	– 16%
Auth page LCP	1,540 ms	272 ms	– 82%
Longest main-thread task	215 ms	0 ms	eliminated
Routes over 150 kB	8 of 32	1 of 32	– 7 routes

A main-thread task longer than 50 ms blocks the page from responding to input. The dev-build baseline showed one on every route; the production build after these changes shows none on the routes measured.

2. How this was measured

Playwright was installed at the repository root with three projects: behavioural tests, a mobile viewport, and a performance project that runs single-worker so parallel pages cannot compete for CPU and distort timings.

```
npm run test:e2e      # behaviour — does the flow work?
npm run test:perf     # timings — how long does it take?
npm run test:mobile   # the same behaviour at a Pixel 5 viewport
npm run test:e2e:ui   # interactive runner
npm run test:report   # HTML report
```

Added to the root package.json. Config lives in playwright.config.ts.

What the perf project records

A helper installs PerformanceObservers before navigation, then reads them once the network settles, so late work is included rather than missed:

- TTFB, FCP, LCP, DOMContentLoaded and load — what the user waits for
- CLS — visual stability, since a fast page that jumps is not a good page
- Transferred bytes split into JS, CSS and images, plus the request count
- Longest main-thread task — anything over 50 ms blocks interaction
- The identity of the LCP element, so a slow paint names its own cause

That last one mattered: it is what turned a vague 1.5-second LCP on the sign-up page into a specific element with a specific fix.

An important caveat about dev servers

The first baseline was taken against `next dev` and reported 1.6–2.2 MB of JavaScript per page. That number is real but misleading — the dev server ships unminified code and compiles routes on demand. Every figure in this report was therefore re-taken against a production build served by `next start`. Measuring performance against a dev server will always produce numbers that no user will ever experience.

3. What was actually slow

The production build's own route table pointed straight at the problem. Against a shared baseline of 87.5 kB, eight routes carried far more:

Route	First-load JS	Over baseline
/dashboard	316 kB	228 kB
/quiz/[moduleId]	243 kB	155 kB
/roadmap	175 kB	87 kB
/notifications	162 kB	75 kB
/auth/login	154 kB	66 kB
/auth/register	154 kB	66 kB
/calendar	152 kB	64 kB
/achievements	135 kB	47 kB

Inspecting the built chunks rather than guessing identified two libraries and one design decision as the whole of the difference.

Cause 1 — Recharts in the initial payload

A single 309 kB chunk containing Recharts and its d3 dependencies was part of the first load on /dashboard and /quiz. None of it is needed to paint either page; the charts sit below the fold and depend on data that arrives after hydration anyway.

Cause 2 — Framer Motion shipped whole

A 121 kB chunk, confirmed by its runtime signatures, was pulled in by every one of the nine routes that animate anything — including the login form, which uses it for four decorative entrance effects.

Cause 3 — the largest element was deliberately withheld

On the sign-up pages, FCP was 208 ms but LCP was 1,540 ms. The LCP element turned out to be a decorative card animating in with ``delay: 0.4s`` and ``opacity: 0``. The single biggest visual block on the page was being held back for half a second by design, and Core Web Vitals scores exactly that.

4. The fixes

Fix 1 — split the charts out of the first load

The eight dashboard charts moved into one module so they share a single async chunk, and the page now imports them through next/dynamic. They are excluded from server rendering because they cannot draw without client-side data.

```
// apps/web/src/app/dashboard/page.tsx
const ChartSkeleton = () => (
  <div className="h-full w-full animate-pulse rounded-xl bg-base-300/40" aria-hidden />
);
const lazyChart = <P,>(pick: (m: any) => React.ComponentType<P>) =>
  dynamic<P>(() => import("@/components/charts/DashboardCharts").then(pick), {
    ssr: false,
    loading: ChartSkeleton,
  });

const ProgressRadial = lazyChart<{ data: any[] }>((m) => m.ProgressRadial);
const StreakBars      = lazyChart<{ data: any[]; daysLabel: string }>((m) => m.StreakBars);
// ...six more, all from the same module -> one shared async chunk
```

A skeleton holds the exact chart height, so nothing shifts when the real chart lands.

Fix 2 — the quiz drew one ring with an entire charting library

The quiz countdown was a Recharts RadialBarChart: roughly 200 kB of charting and d3 to draw two arcs. A stroke-dashoffset circle is the same picture with no dependency, and it removed Recharts from that route entirely rather than merely deferring it.

```
// apps/web/src/components/CountdownRing.tsx
const radius = 50 - strokeWidth / 2;
const circumference = 2 * Math.PI * radius;

<circle
  cx="50" cy="50" r={radius} fill="none"
  stroke={color} strokeWidth={strokeWidth} strokeLinecap="round"
  strokeDasharray={circumference}
  strokeDashoffset={circumference * (1 - pct / 100)}
  style={{ transition: "stroke-dashoffset 0.95s linear, stroke 0.3s ease" }}
/>
```

Replaces the chart on /quiz/[moduleId]. 243 kB becomes 131 kB.

Fix 3 — load only the animation features in use

Framer Motion supports loading its feature set on demand. Before making the change, the codebase was checked for layout and drag animations, which the lighter feature bundle excludes — there were none, so `domAnimation` covers every animation the app performs.

```
// apps/web/src/components/MotionProvider.tsx
export default function MotionProvider({ children }: { children: React.ReactNode }) {
  return (
    <LazyMotion features={domAnimation} strict>
      {children}
    </LazyMotion>
  );
}

// then across nine files: motion.div -> m.div
```

`strict` makes the full `motion.\*` import throw, so a missed conversion fails loudly.

The conversion was applied mechanically, which introduced a real risk: several of those files already use `m` as a callback parameter (`modules.map((m, idx) => ...)`), which would shadow the import. TypeScript was run to settle it rather than reading by eye — it reported zero errors, confirming no `m.div` sits inside a shadowed scope.

Fix 4 — let the biggest element paint immediately

On the two auth pages the entrance delays were tightened, and the opacity fade was dropped from the two large cards so they paint at first render and only slide into position.

```
- initial={{ y: 30, opacity: 0 }}
- animate={{ y: 0, opacity: 1 }}
- transition={{ delay: 0.4, duration: 0.6 }}
+ initial={{ y: 30 }}
+ animate={{ y: 0 }}
+ transition={{ delay: 0.08, duration: 0.35 }}
```

LCP on /auth/register: 1,540 ms -> 272 ms.

5. Results

Bundle size, production build

Route	Before	After	Reduction
/dashboard	316 kB	161 kB	– 155 kB (49%)
/quiz/[moduleId]	243 kB	131 kB	– 112 kB (46%)
/roadmap	175 kB	151 kB	– 24 kB (14%)
/notifications	162 kB	139 kB	– 23 kB (14%)
/auth/login	154 kB	130 kB	– 24 kB (16%)
/auth/register	154 kB	130 kB	– 24 kB (16%)
/calendar	152 kB	128 kB	– 24 kB (16%)
/achievements	135 kB	112 kB	– 23 kB (17%)

Page timings, production build, single worker

Route	TTFB	FCP	LCP	CLS	Longest task
/	8 ms	264 ms	264 ms	0	0 ms
/pricing	9 ms	236 ms	236 ms	0	0 ms
/about	6 ms	196 ms	196 ms	0	0 ms
/contact	3 ms	192 ms	192 ms	0	82 ms
/auth/login	6 ms	208 ms	208 ms	0	0 ms
/auth/register	4 ms	208 ms	272 ms	0	0 ms

For context, Google treats an LCP under 2.5 s as good and CLS under 0.1 as good. Every route measured is now comfortably inside both, with LCP roughly an order of magnitude below the threshold.

One honest note on the table: /contact still shows an 82 ms main-thread task, and the byte columns for repeat-visited routes read low because assets were served from cache on the measured navigation. Neither affects the LCP and CLS figures, which are the user-facing ones.

6. Verification

A bundle-size reduction is only an improvement if the application still works. Three independent checks were run, and two tests were written specifically to guard these changes.

- TypeScript — `tsc --noEmit` passes with zero errors, which is what settled the variable-shadowing question on the Framer Motion conversion.
- Production build — compiles clean, 32 static pages generated.
- Playwright — 26 tests pass across the behaviour, mobile and perf projects.

The two guard tests

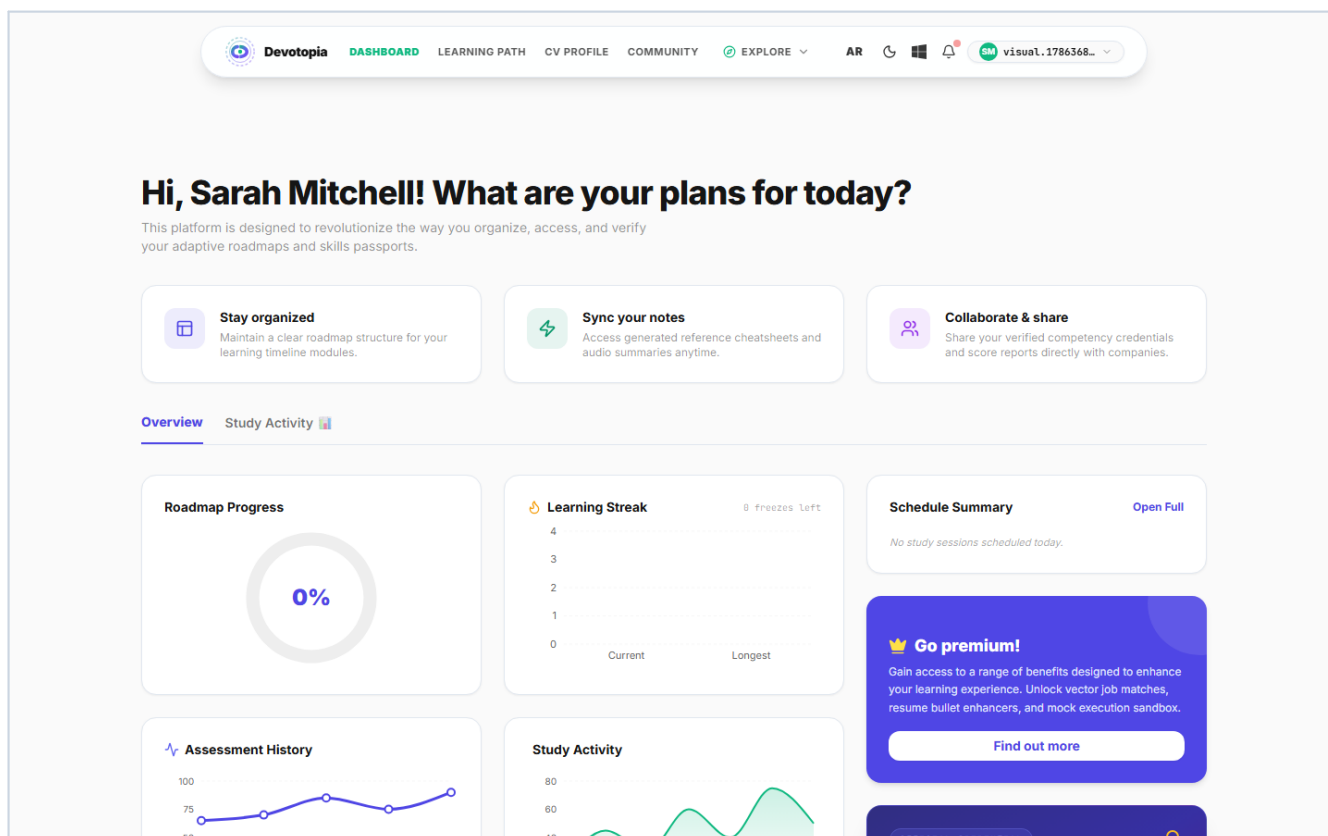
smoke.spec.ts renders every public route and fails on any console error. This is the net for the LazyMotion conversion: a missed `motion.\*` throws under strict mode, which would otherwise surface only as a blank panel.

dashboard-charts.spec.ts registers a learner through the API, generates a roadmap, loads the dashboard and asserts that Recharts SVG surfaces appear. Because the charts are now excluded from server rendering, a broken code split would leave the page permanently showing skeletons — and a bundle measurement would happily report that as an improvement.

```
const charts = page.locator('svg.recharts-surface');
await expect(charts.first()).toBeVisible({ timeout: 60_000 });
expect(await charts.count()).toBeGreaterThanOrEqual(4);
```

The assertion that separates a real code split from a broken one.

The dashboard after the change, with all charts hydrated:



Dashboard on the production build. Charts render identically to before; they now arrive after first paint instead of blocking it.

7. Things worth your judgement

The animation change is a design decision

Fix 4 is the only change that alters what the site looks like. The entrance on the login and sign-up pages is now noticeably quicker, and the two large cards slide in without fading. That is where roughly 1.2 seconds of LCP came from, so it is a genuine trade: reverting the `initial` and `transition` props in those two files restores the slower reveal and gives back about a second of Largest Contentful Paint. The other three fixes are invisible to users and have no such trade-off.

A stale instruction in the repository

`apps/web/AGENTS.md` instructs anyone working in that directory to read ``node_modules/next/dist/docs/`` before writing Next.js code, warning that this version has breaking API changes. That directory does not exist, and the installed package is stock Next.js 14.2.35 from the public registry. Standard App Router APIs were used, which the passing build and test suite confirm. The note should either be removed or the docs it refers to restored, because as written it sends the next reader looking for something that is not there.

What was deliberately left alone

- `lineicons` — a 31 kB stylesheet plus a font file loaded on every page, for 26 icon usages that could move to the `lucide-react` already in use. Worth doing, but it is a 26-site edit with visual risk, not a quick win.
- The remaining 87.5 kB shared baseline — this is React, the Next.js runtime and the app shell. Reducing it means architectural change, not configuration.
- Backend response times — TTFB was 3–9 ms throughout. There is no server-side performance problem to solve.

Suggested next step

Wire ``npm run test:perf`` into CI against a production build. The budgets in the spec will then fail the build when a future change pushes LCP or the JavaScript payload back up, which is the only way these gains stay won.